

ProcRosetta: Training Data Generation

Supplementary Material

ALESSANDRO BERTI, Process and Data Science (PADS), RWTH Aachen University, Germany

WIL M.P. VAN DER AALST, Process and Data Science (PADS), RWTH Aachen University, Germany and Celonis, Germany

1 How the Training Data Was Generated

This supplementary document explains the data-generation pipeline used for the main paper. The goal of the generator is simple: create many small process behaviors, write each behavior in several forms, and keep an explicit record of which forms share the same complete visible-trace language. The model can then learn from paired examples instead of guessing which event log, process tree, and Petri net belong together. This is the generator’s chosen equivalence semantics; it does not imply equality under every structural or partial-order semantics.

Figure 1 summarizes the six stages detailed below.

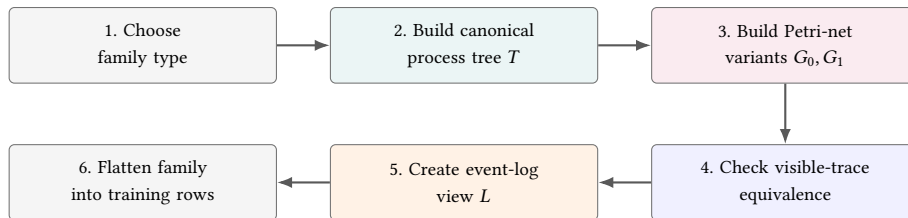


Fig. 1. The behavior-family data-generation pipeline. Equivalence is checked before an event-log view is created and the family is flattened into rows.

1.1 What One Training Row Contains

Each row in the generated data is a *process sample*. A row contains four main fields:

- (1) a canonical process tree T ;
- (2) a generated event-log view L , stored as a finite list of visible activity traces;
- (3) a Petri-net graph G , stored as typed nodes, labeled transitions, directed arcs, and initial/final marking vectors;
- (4) an equivalence identifier e , shared by all rows in the same visible-trace equivalence family.

The process tree is the decoder target. The generated event log is the finite trace-based view of that family. The Petri-net graph is its graph-based view. During training, rows with the same equivalence identifier are treated as positive matches for the contrastive objective.

The persisted JSONL rows also store bookkeeping fields: `model_variant_id`, `log_view_id`, `representation_kind`, `equivalence_level`, and `metadata`. These fields make it possible to inspect which Petri-net variant was used, which generator motif produced the family, and which equivalence checks were passed.

Authors’ Contact Information: [Alessandro Berti](mailto:a.berti@pads.rwth-aachen.de), Process and Data Science (PADS), RWTH Aachen University, Aachen, Germany, a.berti@pads.rwth-aachen.de; [Wil M.P. van der Aalst](mailto:wvdaalst@pads.rwth-aachen.de), Process and Data Science (PADS), RWTH Aachen University, Aachen, Germany and Celonis, Munich, Germany, wvdaalst@pads.rwth-aachen.de.

1.2 Current Split and Generator Settings

The current data directory was generated with the behavior-family generator, fixed seed 13, and two representation rows per behavior family. The split sizes are:

Split	Rows	Behavior families
Training	8,192	4,096
Validation	1,024	512
Test	1,024	512

The important generator settings are:

- maximum ordinary-process-tree recursion depth: 8;
- minimum ordinary-process-tree depth/size targets: depth 4 and 20 nodes;
- minimum distinct activities per ordinary process tree: 8;
- maximum activity labels: 30;
- maximum operator arity: 3;
- traces per generated event-log view: 128;
- activity-label reuse probability: 0.15;
- early leaf probability during process-tree recursion: 0.55;
- representation rows per behavior family: 2;
- event-log views per behavior family: 1;
- controlled-motif context suffix: 6 fresh activities;
- family-type weights: 0.55 for ordinary process trees and 0.15 for each of the three controlled motifs;
- strict minimum motif coverage: 8 families per motif in training and 4 per motif in validation and test;
- enabled process-tree operators: SEQ, XOR, AND, and two-child LOOP.

The LOOP operator is generated in the ordinary-process-tree family. The generated and decoded loop form is the two-child form $\text{LOOP}(T_{body}, T_{redo})$; the tokenizer mask therefore emits ARITY_2 after a LOOP token. Because a loop can repeat an arbitrary number of times, ordinary loop families use finite process-tree playout to construct the observed trace pool rather than trying to enumerate an unbounded language exhaustively.

1.3 Why the Generator Uses Behavior Families

The generator does not create isolated triples one by one. Instead, it creates *behavior families*. A behavior family combines one canonical process tree with two Petri-net representations certified to have the same complete visible-trace language (or related directly by isomorphic renaming for ordinary loop families). Both rows in a family share:

- the same canonical process tree;
- the same generated trace list;
- the same equivalence identifier.

They differ in their Petri-net graph. Sometimes the second graph is only a renamed copy of the first graph. In other cases, the second graph uses a different internal structure while keeping the same complete visible traces. For example, the concurrency/interleaving family has the same linearized traces but deliberately differs in partial-order semantics, recorded as `partial_order_equivalent=false`. Thus the family identifier denotes the explicit visible-trace equivalence used for training, not equivalence under all behavioral notions.

The current configuration favors ordinary process trees while retaining all three controlled family types. With the split sizes above, the deterministic weighted quota allocator produces:

Family type	Training families	Validation families	Test families
Ordinary process tree	2,243	277	277
Duplicate label vs. silent routing	618	78	78
Concurrency vs. explicit interleaving	618	79	79
Non-free-choice M pattern	617	78	78

1.4 Step 1: Choose the Family Type

For each split, the generator first computes how many behavior families are needed. Because the current configuration stores two rows per behavior family, 8,192 training rows become 4,096 training families. The same calculation turns 1,024 validation rows and 1,024 test rows into 512 families each.

The ordinary-process-tree family has weight 0.55; the duplicate-label/silent-routing, concurrency/interleaving, and non-free-choice families have weight 0.15 each. The generator first reserves the strict per-type minimum and then allocates the remaining families proportionally by the largest-remainder method. It shuffles the resulting deterministic quota plan using a seed derived from the global seed, the split name, and the phrase `motif-quota-plan`. This gives a mixed order on disk while preserving the planned weighted counts and minimum coverage.

The current run uses strict class coverage. Before replacing the data directory, the generator verifies that every requested row count is divisible by the number of rows per family and that the requested family count can meet the per-motif minimum. An infeasible request therefore fails before the existing split files are removed. A separate `best_effort` mode is available for small diagnostic datasets; it records motif and representation-slot deficits instead of requiring them to be zero.

A complete behavior family is assigned to exactly one split, and its equivalence identifier includes that split. The generator does not, however, deduplicate complete visible languages across different families or splits. Consequently, family members cannot leak across a split boundary, but equal or near-equal behaviors can still occur independently in training and evaluation.

1.5 Step 2: Build the Canonical Process Tree

The canonical process tree is the target representation used by the decoder. There are two cases.

Ordinary-process-tree families. For ordinary-process-tree families, the generator samples a random process tree. It first samples a pool of 8 to 30 activity labels, then recursively builds the process tree:

- (1) If the maximum depth has been reached, it creates an activity leaf.
- (2) Otherwise, with probability 0.55 it also stops early and creates an activity leaf.
- (3) If it does not stop, it chooses one operator from SEQ, XOR, AND, and LOOP.
- (4) For SEQ, XOR, and AND, it samples an arity between 2 and 3 and recursively generates the children.
- (5) For LOOP, it recursively generates exactly two children: the body and the redo branch.

Activity labels are introduced as `a0`, `a1`, and so on. With probability 0.15 the generator reuses an earlier label instead of introducing a new one. During construction, labels are normalized to `A0`, `A1`, and so on in depth-first tree order.

Children of XOR and AND nodes are sorted by a canonical structural key, so equivalent child permutations do not become different targets. The final stored labels are assigned later from the event-log view (Section 1.9).

The sampler makes up to 100 attempts and keeps the first process tree that contains a loop, has at least eight distinct activities, reaches the configured minimum depth, and has at least 20 nodes. Candidate scoring prioritizes loop presence, then progress toward the eight-activity floor, depth, and size. If no candidate meets every selection target, the best candidate is returned after fresh activities are added sequentially until it has at least eight distinct labels. The alphabet floor is therefore a hard postcondition; loop presence, depth, and size remain selection targets rather than hard postconditions.

Figure 2 separates the recursive construction rule from the candidate-selection rule.

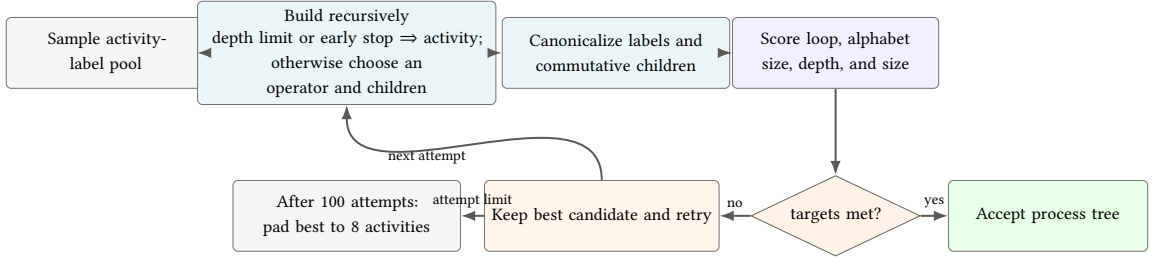


Fig. 2. Ordinary-process-tree sampling. Loop presence, eight distinct activities, depth, and size guide selection. The final fallback pads the highest-scoring candidate when necessary, making only the alphabet floor a hard postcondition.

Controlled behavior families. The other three family types use small hand-defined canonical process trees:

- duplicate-label vs. silent-routing families use $\text{XOR}(\text{SEQ}(A0, A1), \text{SEQ}(A0, A2))$;
- concurrency vs. explicit-interleaving families use $\text{AND}(A0, A1)$;
- non-free-choice M-pattern families use $\text{XOR}(A0, \text{AND}(A1, A2))$.

These process trees are intentionally small because their purpose is to isolate one specific representation issue at a time. To avoid making every controlled case too bare, the generator appends up to six fresh activity labels as a shared sequential suffix. The suffix is added as nested binary sequence nodes, so the configured maximum arity remains 3. For example, if a motif already uses $A0$, $A1$, and $A2$, the suffix can add $A3$ through $A8$. The same suffix is added to both Petri-net variants in the family.

1.6 Step 3: Build Two Petri-Net Variants

Each behavior family produces exactly two accepting Petri nets, $N_v = (G_v, M_v^0, M_v^f)$ for $v \in \{0, 1\}$. The graph G_v contains places, transitions, and directed arcs; M_v^0 and M_v^f are its initial and final markings. The two variants need not have the same nodes or topology. Instead, the controlled families are constructed to satisfy

$$\mathcal{L}_{\text{vis}}(N_0) = \mathcal{L}_{\text{vis}}(N_1), \quad (1)$$

where $\mathcal{L}_{\text{vis}}$ contains complete firing sequences after silent τ transitions have been removed. Step 4 verifies this invariant. For ordinary loop families, whose languages are unbounded, a graph-isomorphism certificate replaces enumeration. Step 6 later turns N_0 and N_1 into two rows while keeping the same process tree, event-log view, and family identifier.

Figures 3–6 use standard Petri-net notation: circles are places, a dot marks the initial token, a double circle marks the final place, labeled rectangles are visible transitions, and gray τ rectangles are silent transitions. The figures show the core motif; the identical sequential context suffix is omitted.

Ordinary-process-tree families: isomorphic renaming. Variant N_0 is the PM4Py conversion of the sampled process tree. Variant N_1 is a deep copy with bijections ϕ_P over places and ϕ_T over transitions. The copy preserves every arc and weight, every visible label, and the initial and final markings; only internal node identifiers change. In particular, (x, y) is an arc in N_0 exactly when $(\phi(x), \phi(y))$ is an arc in N_1 . Figure 3 shows this operation on a small sequential example. The actual generated process tree may be much larger, but the renaming rule is identical.

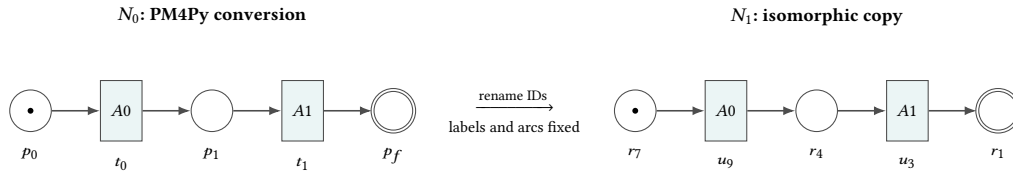


Fig. 3. Isomorphic renaming for an ordinary-process-tree family. Place and transition identifiers change, but topology, visible labels, and markings do not.

Because isomorphism already proves behavioral equivalence, this pair primarily teaches invariance to arbitrary serialization names. For ordinary loop families it also gives a finite certificate without attempting to enumerate an infinite visible language.

Duplicate-label vs. silent-routing families: move the choice. Both variants implement $\text{XOR}(\text{SEQ}(A0, A1), \text{SEQ}(A0, A2))$. In N_0 , the initial place has two outgoing visible transitions, both labeled $A0$. Firing one of them simultaneously selects the $A1$ or $A2$ branch. In N_1 , a single $A0$ transition fires first; only afterward do two competing silent transitions route the token to $A1$ or $A2$. Thus the location and visibility of the choice differ, and the number of $A0$ transitions differs, but both variants have exactly the visible language $\{\langle A0, A1 \rangle, \langle A0, A2 \rangle\}$.

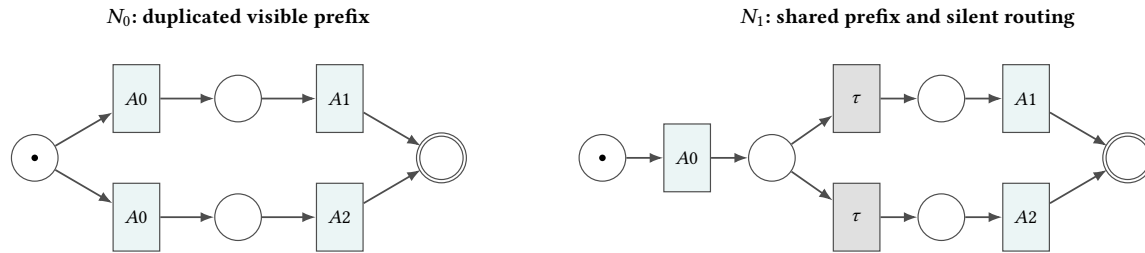


Fig. 4. The duplicate-label and silent-routing variants. The choice is made while firing one of two $A0$ transitions in N_0 , but after the single $A0$ transition through silent routing in N_1 .

Concurrency vs. explicit-interleaving families: change the causal structure. Both variants realize the complete visible traces $\langle A0, A1 \rangle$ and $\langle A1, A0 \rangle$. The PM4Py-derived variant N_0 uses a silent split to create two tokens. Consequently, $A0$ and $A1$ are enabled together and may overlap; a silent join waits for both. The hand-built variant N_1 never holds two concurrent tokens. Instead, its upper serial path spells out $A0$ then $A1$, and its lower path spells out

A1 then A0. The event sequences agree, but the partial-order semantics do not, so the generator explicitly records `partial_order_equivalent=false` for N_1 .

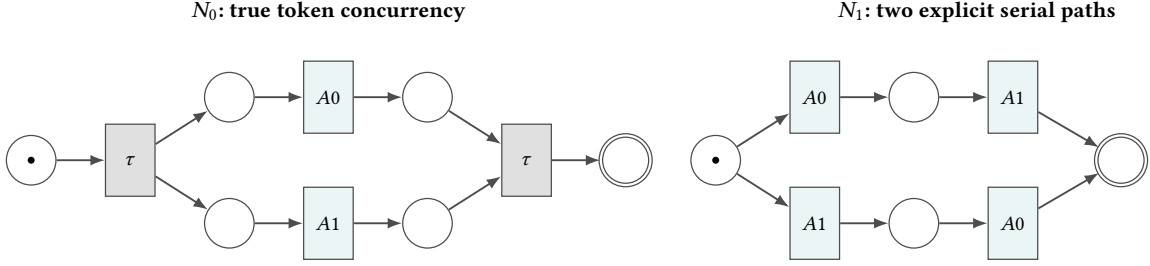


Fig. 5. True concurrency and explicit interleaving have the same two complete visible traces. Only N_0 represents A0 and A1 as simultaneously enabled.

Non-free-choice M-pattern families: change the conflict structure. The canonical process tree is $\text{XOR}(A0, \text{AND}(A1, A2))$. In the block-structured variant N_0 , the first choice is global: either A0 fires, or a silent split selects the parallel A1/A2 branch. In the hand-built M-pattern N_1 , a silent split first produces tokens in places p_1 and p_2 . Transition A0 requires both tokens, while A1 competes for the token in p_1 and A2 competes for the token in p_2 . Hence A0 shares an input place with each of A1 and A2 but does not have the same preset, violating the free-choice condition. Nevertheless, both variants complete exactly $\langle A0 \rangle$, $\langle A1, A2 \rangle$, and $\langle A2, A1 \rangle$.

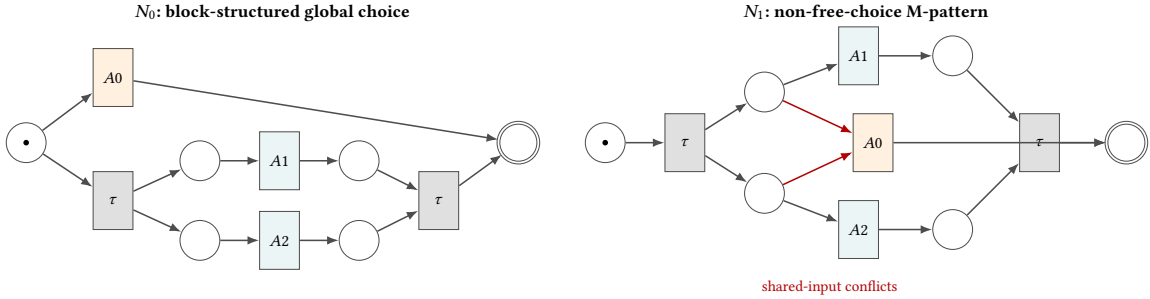


Fig. 6. A block-structured realization and a non-free-choice M-pattern. The red arcs highlight the two inputs required jointly by A0 in N_1 .

For every controlled family, the generator then appends each context activity to both final markings in the same order, creating a new final place after each activity. This shared suffix lengthens all complete traces equally and does not alter the structural contrast illustrated above.

1.7 Step 4: Check Behavioral Equivalence

For every family not handled by the ordinary-loop isomorphism shortcut, the generator compares the visible trace sets returned for all Petri-net variants. The checker explores firing sequences and records search states comprising the marking, visible prefix, and firing count. It records traces that reach the final marking, omits invisible transitions, and reports whether exploration finished or a resource bound truncated it.

The current check uses these bounds:

- a threshold of 5,000 visited search states;
- a threshold of 10,000 distinct complete visible traces;
- at most 32 visible activities per trace;
- at most $4 \cdot 32 + |T|$ transition firings along a search path, where $|T|$ is the number of Petri-net transitions.

The implementation flags the search as incomplete when either count threshold is exceeded or either path bound prevents further exploration. The trace set returned up to that point must be nonempty and identical for every variant. If all searches finish, the certificate status is `exact`; if a bound truncates at least one search, the compared finite result receives status `bounded`. The current configuration sets `exact_equivalence_only_for_training` to `false`, so a bounded family is eligible for any split. In the persisted data used here, however, every controlled family received an exact certificate; there are no bounded rows.

Ordinary loop families are handled separately because their language is unbounded. Their second Petri net is an isomorphic clone of the PM4Py process-tree conversion, so equivalence is certified structurally by the renaming transformation. All ordinary-process-tree rows in the persisted data therefore store the status `isomorphic`. Each accepted row includes the certificate status, semantics, checked variant names, and reference-language size in its metadata.

The generator also checks simple structural health conditions. In particular, the final marking must be reachable and there must be no dead transitions. A family that fails these checks is rejected.

1.8 Step 5: Create the Generated Event-Log View

After equivalence is checked or certified, the generator creates a trace pool. For controlled families in the current data, the pool is the sorted exact visible language returned by the completed checker. For ordinary loop families, PM4Py performs $\max(256, 4n)$ process-tree playouts for requested event-log size n ; the generator then deduplicates and sorts the observations. Thus the current $n = 128$ run draws 512 playout traces before deduplication. It creates one event-log view per family and stores 128 traces in that view.

The current configuration lists `uniform_variants` followed by `resampled`. Because it requests only one event-log view, that view uses the first mode. In `uniform_variants`, the generator cycles through the trace pool until it has 128 traces, then shuffles them with a deterministic seed. If a behavior has only two variants, both recur; if the pool has more than 128 variants, only its first 128 sorted entries are used before shuffling.

The generator also supports sparse, long-tail, incomplete, resampled, and noisy modes for other experiments. The persisted split uses one clean `uniform_variants` view per family; all stored trace-edit lists are empty.

Family construction, quota order, and event-log-view shuffling use stable seeds; loop trace pools are an exception: although a derived playout seed is recorded, the current call does not apply it to PM4Py's playout randomness. Exact loop trace pools are therefore not guaranteed by the generator seed alone and can depend on ambient random state as well as the installed PM4Py version. The persisted JSONL rows, rather than regeneration from seed alone, define the exact loop event-log contents used in the study.

1.9 Step 6: Flatten Families into Training Rows

The final step turns each behavior family into rows. With the current configuration, each family has one generated event-log view and two Petri-net variants, so it becomes exactly two rows:

- (process tree, same event-log view, Petri-net variant 0, e),
- (process tree, same event-log view, Petri-net variant 1, e).

Before these rows are stored, the generator scans the event-log view in its persisted trace order and assigns A_0 , A_1 , and so on when each activity first appears. Family-alphabet labels not observed in that finite view are appended in stable tree/trace-pool order. The resulting map is applied to the process tree, every trace event, and every visible Petri-net transition; trace-edit provenance is relabeled by the same map. Consequently, all modalities in a row use exactly the label scheme that an external event log receives at inference time. The row records `label_scheme=first_seen_per_log_view` in its metadata.

Figure 7 makes the shared and variant-specific fields explicit.

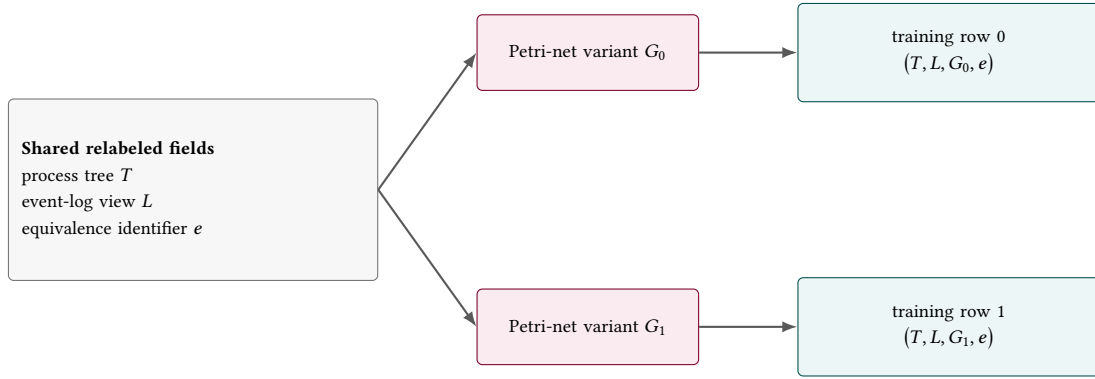


Fig. 7. Flattening one behavior family into two training rows after applying the event-log view’s first-seen activity map. The process tree, event-log view, and equivalence identifier are duplicated; only the Petri-net graph changes.

Both rows have the same equivalence identifier e . The process tree and trace list are identical in the two rows. The Petri-net graph is different.

Rows are written to:

- `data/training/samples.jsonl`;
- `data/validation/samples.jsonl`;
- `data/test/samples.jsonl`.

The generator also writes `data/metadata.json`, which records the seed, the full synthetic configuration, split sizes, motif counts, representation counts, average process-tree and trace statistics, and maximum Petri-net graph sizes. For each split it records both planned and actual motif quotas, per-motif representation-slot counts, any strict-coverage deficits, and whether the minimum-coverage check passed. During generation, `sample.py` reports completed flattened rows with a per-split progress bar on standard error; `-quiet` disables this display without changing the generated data.

1.10 How the Row Is Used During Training

At training time, each row is converted into tensors:

- the process tree becomes a prefix token sequence with explicit arity tokens, preserving its stored first-seen activity labels;
- the generated traces become padded activity-token sequences;
- the Petri net becomes node-type indices, transition-label ids, marking features, and two adjacency matrices.

The batch collator also creates a positive-pair mask from the equivalence identifiers. If two rows have the same equivalence identifier, they are treated as matching views of the same visible-trace equivalence family. Training batches can also apply a consistent random renaming of activity labels within a family. This preserves the behavior while reducing dependence on the literal label names.

In short, the data-generation pipeline makes the supervision explicit. The model sees a process tree, a finite generated event log, and a Petri-net graph; it also sees which rows belong to the same visible-trace equivalence family. The generated families therefore train the model to align artifact types by behavior rather than by node names, transition counts, or one particular Petri-net structure.